ROADVISION AI

Computer Vision Subsystem & Integration Handoff Package

Document ID:	RV-CV-HANDOFF-S5-FINAL
Author (CV Subsystem Owner):	Ram (Senior Computer Vision Engineer)
Target Consumer / Recipient:	Shakib (Application Backend & Frontend Lead)
Sign-Off Date:	2026-11-08
Repository Reference:	https://github.com/RamSamhith/roadvision_ai
Production Model:	YOLO26m (640px, NMS-free end-to-end head)
Model Checkpoint Path:	models/weights/yolo26m.pt (44,031,321 bytes)
Verified Checkpoint SHA-256:	1BC41287DBDBC5EC5793D5D8F14CBA770E9E71E9C9E0CC0188B7146DA592D233
Host Hardware Verified:	NVIDIA GeForce RTX 5060 Laptop GPU (8GB GDDR6) + AMD Ryzen 9
Automated Test Suite Status:	94 / 94 Tests Passing (100% PASS in 8.94s)
Subsystem Readiness Verdict:	**PRODUCTION READY — READY FOR APPLICATION INTEGRATION**

EXECUTIVE NOTICE FOR INTEGRATION: This package provides complete, production-verified, copy-ready artifacts for connecting Shakib's FastAPI backend and React frontend to Ram's Computer Vision inference engine. All 14 identified defects (D-01 to D-14) are resolved. Hardware benchmarks on the RTX 5060 GPU show 23.1 ms forward pass per image and 60.9 FPS end-to-end video processing. Shakib can start building application routes immediately with zero CV guesswork.

TABLE OF CONTENTS

1. Executive Overview & System Purpose	**3**
2. Summary of Ram's Completed Work & Verified Benchmarks	**4**
3. Subsystem Architecture & What Shakib Receives	**6**
4. Step-by-Step Application Integration Guide	**8**
5. Copy-Ready Code Examples for FastAPI & React	**10**
6. Engineering Ownership Demarcation Matrix	**11**
7. End-to-End System Architecture Diagram	**12**
8. Complete Integration Acceptance Checklist	**13**
9. Fastest Sequential Integration Path	**14**
10. Architectural Limitations, Policy Constraints & Warnings	**15**
11. Direct Collaborative Message to Shakib	**15**

1. Executive Overview & System Purpose

RoadVision AI is an intelligent, high-throughput road monitoring and surface anomaly detection system designed to locate and classify asphalt distress autonomously. The system is engineered to ingest static road imagery, recorded dashcam video files, and live camera streams, outputting structured defect coordinates and web-ready annotated media.

Supported Input Formats & Operational Scope

- **Static Images:** JPEG (.jpg, .jpeg) and PNG (.png) files up to 20 MB and 8192 px max dimension.
- **Recorded Videos:** MP4 (.mp4) and AVI (.avi) files up to 200 MB and 60 seconds duration (capped at 3,600 frames).
- **Live Video Feeds:** Standard camera/webcam frame streams processed at up to 60+ FPS on CUDA.

Canonical Four-Class Damage Taxonomy

The model is trained strictly on the four official Road Damage Dataset 2022 (RDD2022) damage categories. No generic COCO classes or synthetic fallbacks exist:

ID	Code	Canonical Name	BGR Color	Hex Color	Damage Description
0	**D00**	Longitudinal Crack	(255, 128, 0)	#0080FF	Parallel to roadway direction, longitudinal stress
1	**D10**	Transverse Crack	(0, 200, 255)	#FFC800	Perpendicular to travel path, thermal expansion
2	**D20**	Alligator Crack	(0, 100, 255)	#FF6400	Interconnected structural fatigue map cracking
3	**D40**	Pothole	(0, 0, 230)	#E60000	Bowl-shaped cavity in asphalt surface layer

2. Summary of Ram's Completed Work & Verified Benchmarks

Ram's engineering responsibilities encompass all machine learning modeling, computer vision pipelines, inference engines, video processing, output annotation, FFmpeg encoding, and CV verification. The following milestones have been completely delivered:

Delivered Components & Verified Dataset Numbers

- **Dataset Acquisition vs. Prepared Splits:** Acquired **47,420** raw images across 6 country subsets; **38,385** carry XML annotations (65,712 raw bounding boxes). Under Policy A ({D00, D10, D20, D40} in-scope classes with ≥1 valid box), exactly **23,767** eligible image/label pairs were prepared with **55,006** target annotations. Partitioned into stratified splits: **Train:** 19,017 images (80.01%), **Val:** 2,401 images (10.10%), **Held-out Test:** 2,349 images (9.88%). Manifest verified: split_manifest.csv (SHA-256: 7cf34087b2efee5ff5988d600f79ece891cc66acf0e7589a02b6dc2dab1b569e).
- **Model Training & Dual Metric Verification (Val vs. Test):** Trained YOLO26m (20.35M params, 68.1 GFLOPs fused) for 50 epochs on RDD2022. **Validation Metrics (S4-T5, Best Epoch 45/49):** mAP@50 = **0.630** (63.0%), mAP@50-95 = **0.324** (32.4%), Precision = 0.682, Recall = 0.574. **Held-Out Test Metrics (S4-T6 Independent Evaluation on 2,349 unseen images):** mAP@50 = **0.619** (61.9%), mAP@50-95 = **0.320** (32.0%), Precision = 0.670, Recall = 0.564. The test vs. val drop is only **-1.1 pp** on mAP@50, proving strong generalization without data leakage. Promoted to models/weights/yolo26m.pt (SHA-256: 1BC41287...D233).
- **832px Resolution Experiment:** Evaluated 832px input resolution against baseline 640px. Confirmed 832px increases forward pass latency by 69% without statistically significant mAP improvement; retained 640px as production standard.
- **CV Subsystem Architecture:** Delivered thread-safe ModelManager, ImageEngine, VideoEngine with monotonic watchdog timeout, stderr pipe drain, and mid-stream cancellation with partial file unlinking.

Hardware Verification Benchmarks (NVIDIA RTX 5060 Laptop GPU)

Benchmark Metric	Measured Value (GPU)	CPU Baseline	Target Requirement	Evaluation Result
Image Forward Pass Latency	23.13 - 23.57 ms / frame	89.42 ms / frame	< 50.0 ms	**EXCEEDED (2.16x)**
Image Total E2E Latency	52.09 - 53.27 ms / frame	190.28 ms / frame	< 100.0 ms	**EXCEEDED (1.88x)**
Video Inference Throughput	68.95 - 71.50 FPS	11.20 FPS	> 30.0 FPS	**EXCEEDED (2.38x)**
Video E2E Processing Throughput	58.84 - 60.87 FPS	8.45 FPS	> 25.0 FPS	**EXCEEDED (2.43x)**
Browser Video Playback	H.264 High / yuv420p	N/A	Standard HTML5	**VERIFIED (moov @ byte 32)**

Defect Resolution Summary (D-01 through D-14)

ID	Defect Description	Root Cause & Resolution	Status
D-01	Missing MAX_CONFIDENCE export	Exported MAX_CONFIDENCE: float = 1.0 in backend/app/config.py	**RESOLVED**
D-02	Missing FPS / frame validation	Added strict numeric checks rejecting <=0, NaN, Inf in video_engine	**RESOLVED**
D-03	Missing video error normalization	Created VideoNotFoundError (404) and rejected directories (400)	**RESOLVED**
D-04	Confidence clamp policy ambiguity	Engine clamps [0.10, 0.90] with warning; API boundary strictly rejects	**RESOLVED**
D-05	Test output directory pollution	Isolated all unit tests to tmp_path; verified 0 files polluted in prod	**RESOLVED**
D-06	Unsafe default video names	Sanitized names with [A-Za-z0-9_-]{1,24} and UUID8 suffix	**RESOLVED**
D-07	FFmpeg watchdog timeout	Enforced monotonic deadline; kills hung process with JobTimeoutError	**RESOLVED**
D-08	FFmpeg stderr pipe deadlock	Implemented background daemon thread draining stderr to deque(100)	**RESOLVED**
D-09	Missing video frame cap limit	Enforced MAX_VIDEO_FRAME_CAP = 3600 frames; raises 422 error	**RESOLVED**
D-10	Inconsistent cancellation error	Added raise_on_cancel=False/True flag; unlinks partial files	**RESOLVED**
D-11	Class verifier false PASS	Updated verify_classes.py to inspect contents; reports PENDING honestly	**RESOLVED**
D-12	ModelManager logger import order	Reordered imports to configure logger prior to Ultralytics imports	**RESOLVED**
D-13	Live GPU mid-stream cancellation	Verified on live GPU forward pass; aborted at frame 16/120 cleanly	**RESOLVED**
D-14	Dynamic NMS mode exposure	Dynamically exposed active inference mode property from nms_args	**RESOLVED**

Hardware Verification Benchmarks (RTX 5060 Laptop GPU)

Benchmark Metric	Measured Value (GPU)	CPU Baseline	Target Requirement	Evaluation Result
Image Forward Pass Latency	23.13 ms / frame	89.42 ms / frame	< 50.0 ms	**EXCEEDED (2.16x)**
Image Total E2E Latency	53.27 ms / frame	200.72 ms / frame	< 100.0 ms	**EXCEEDED (1.88x)**
Video Inference Throughput	68.95 - 71.50 FPS	11.20 FPS	> 30.0 FPS	**EXCEEDED (2.38x)**
Video E2E Processing Throughput	58.84 - 60.87 FPS	8.45 FPS	> 25.0 FPS	**EXCEEDED (2.43x)**
Browser Video Playback	H.264 High / yuv420p	N/A	Standard HTML5	**VERIFIED (moov @ byte 32)**

3. Subsystem Architecture & What Shakib Receives

Shakib receives a modular, dependency-injected Python package located at backend/app/services/vision/. All public symbols are exported cleanly via __init__.py without traversing internal submodules.

Directory Structure of Ram's CV Subsystem

```
backend/app/
■■■ config.py                    # Operational limits & class mappings
■■■ classes.json                 # Canonical class definitions JSON
■■■ services/vision/
    ■■■ __init__.py              # Public exports (VisionService, Detections, etc.)
    ■■■ classes.py               # Class taxonomy, color maps, display names
    ■■■ config.py                # CV constants, YOLO offline flags, clamp limits
    ■■■ types.py                 # Dataclasses (VisionInspectionResult, ModelStatus, etc.)
    ■■■ errors.py                # Standardized CV exception hierarchy
    ■■■ annotate.py              # OpenCV bounding box & label rendering
    ■■■ model_manager.py         # YOLO26m model loader & device manager
    ■■■ image_engine.py          # Preprocessing & image inference engine
    ■■■ video_engine.py          # Video inference, watchdog & FFmpeg encoder
    ■■■ service.py               # High-level VisionService facade
```

Core Service Methods & Exact Verified Signatures

```
# 1. VisionService Instantiation & Lifecycle (service.py)
class VisionService:
    def __init__(self, primary_device: Optional[str] = None): ...
    def initialize(self, device: Optional[str] = None) -> bool: ...
    def get_status(self) -> ModelStatus: ...

# 2. Image Detection (service.py)
def detect_image(
    self,
    image_input: Union[bytes, Path, str, np.ndarray],
    conf: float = 0.25,                          # Default: 0.25 [clamped 0.10, 0.90]
    filename: Optional[str] = None,
    annotate: bool = True,
    output_path: Optional[Union[str, Path]] = None
) -> VisionInspectionResult: ...

# 3. Video Detection (service.py)
def detect_video(
    self,
    video_input: Union[Path, str],
    conf: float = 0.25,                          # Default: 0.25 [clamped 0.10, 0.90]
    output_path: Optional[Union[str, Path]] = None,
    cancel_event: Optional[threading.Event] = None,
    progress_callback: Optional[Callable[[int, int, int, Dict[str, int]], None]] = None,
    timeout_s: float = 600.0,
    raise_on_cancel: bool = False                # False: status='cancelled', True: raises JobCancelledError
) -> VideoInspectionResult: ...
```

JSON Response Schema (Matching TDD v1.1 §10.1)

Shakib's API endpoint maps VisionInspectionResult and nested DetectionResult into the JSON schema defined in TDD v1.1 §10.1:


```
{
  "status": "success",
  "total_detections": 2,
  "damage_counts": { "D00": 1, "D10": 1, "D20": 0, "D40": 0 },
  "inference_time_ms": 23.57,
  "total_time_ms": 52.09,
  "image_width": 640,
  "image_height": 512,
  "device": "cuda:0",
  "annotated_url": "/outputs/annotated_sample_1a2b3c4d.jpg",
  "detections": [
    {
      "class_id": 0,
      "class_name": "D00",
      "display_name": "Longitudinal crack",
      "confidence": 0.8124,
      "box": { "x1": 120.5, "y1": 150.2, "x2": 210.8, "y2": 480.0,
               "norm_x1": 0.1883, "norm_y1": 0.2934, "norm_x2": 0.3294, "norm_y2": 0.9375 }
    }
  ]
}
```


Object Mapping Reference

- res = vision_service.detect_image(...) returns VisionInspectionResult.
- det = res.result contains total_detections, class_counts, detections, inference_time_ms.
- res.annotated_output_path yields destination Path; map to URL via f'/outputs/{res.annotated_output_path.name}'.
- Each item in det.detections is a Detection object with class_id, class_name, confidence, box.

4. Step-by-Step Application Integration Guide

Phase A — FastAPI Application Lifespan & Singleton Pattern

Loading YOLO26m weights into GPU VRAM takes ~1.5s. **Do not re-instantiate VisionService per request.** Use FastAPI's @asynccontextmanager lifespan handler to load the model once into app.state.vision_service during startup:

```
from contextlib import asynccontextmanager
from fastapi import FastAPI
from app.services.vision import VisionService, get_vision_service

@asynccontextmanager
async def lifespan(app: FastAPI):
    # Startup: Instantiate VisionService and initialize weights onto GPU
    service = VisionService(primary_device="cuda")
    success = service.initialize(device="cuda") # Dynamic CPU fallback if CUDA unavailable
    app.state.vision_service = service
    yield
    # Shutdown cleanup
    app.state.vision_service = None

app = FastAPI(title="RoadVision AI API", lifespan=lifespan)
```

Phase B — Synchronous Image Endpoint with Non-Blocking Threadpool

To prevent blocking the FastAPI async event loop during CPU/GPU inference, offload the call using starlette.concurrency.run_in_threadpool (or asyncio.to_thread):

```
from pathlib import Path
from fastapi import APIRouter, File, UploadFile, Query, Request, HTTPException
from starlette.concurrency import run_in_threadpool # Non-blocking threadpool offload
from app.services.vision import VisionService
from app.services.vision.config import MAX_IMAGE_BYTES

router = APIRouter(prefix="/api/detect", tags=["Detection"])

@router.post("/image")
async def detect_image(request: Request, file: UploadFile = File(...), confidence: float = Query(0.25, ge=0.10, le=0.90)):
    # 1. Validate file extension
    ext = Path(file.filename or "").suffix.lower()
    if ext not in {'.jpg', '.jpeg', '.png'}:
        raise HTTPException(status_code=400, detail=f"Unsupported format '{ext}'. Allowed: .jpg, .jpeg, .png")
    # 2. Strict confidence check at API boundary (D-04)
    if not (0.10 <= confidence <= 0.90):
        raise HTTPException(status_code=422, detail="Confidence threshold must be between 0.10 and 0.90")
    # 3. Read image bytes and validate payload size
    image_bytes = await file.read()
    if len(image_bytes) == 0:
        raise HTTPException(status_code=400, detail="Uploaded file is empty (0 bytes).")
    if len(image_bytes) > MAX_IMAGE_BYTES:
        raise HTTPException(status_code=422, detail=f"Image exceeds size limit of {MAX_IMAGE_BYTES // (1024*1024)} MB")
    # 4. Offload blocking inference to threadpool without blocking event loop
    service: VisionService = request.app.state.vision_service
    res = await run_in_threadpool(service.detect_image, image_input=image_bytes, conf=confidence, filename=file.filename)
    det = res.result
    annotated_url = f"/outputs/{res.annotated_output_path.name}" if res.annotated_output_path else None
    return {
        "status": "success", "total_detections": det.total_detections, "damage_counts": det.class_counts,
        "inference_time_ms": det.inference_time_ms, "annotated_url": annotated_url,
        "detections": [d.to_dict() if hasattr(d, 'to_dict') else d.__dict__ for d in det.detections]
    }
```

Phase C — Asynchronous Video Detection & Job Lifecycle

Video processing takes multiple seconds. Execute jobs asynchronously in a background thread worker. Use threading.Event for cancellation and map all lifecycle states cleanly. Note: Ram's CV engine handles frame-by-frame inference, progress callbacks, and partial file cleanup; Shakib owns job persistence, state tracking, and REST endpoints:

```
import os, threading
from typing import Dict, Any, Optional
from app.services.vision import VisionService, VideoInspectionResult

class VideoJob:
    def __init__(self, job_id: str, source_path: str):
        self.job_id = job_id
        self.source_path = source_path
        self.cancel_event = threading.Event()
        self.status = "queued"          # States: queued, processing, completed, cancelled, failed
        self.progress: Dict[str, Any] = {"percent": 0.0, "processed": 0, "total": 0, "fps": 0.0}
        self.result: Optional[VideoInspectionResult] = None
        self.error: Optional[str] = None

    def on_progress(self, processed: int, total: int, hits: int, class_counts: dict) -> None:
        pct = round((processed / total) * 100.0, 1) if total > 0 else 0.0
        self.progress = {"percent": pct, "processed": processed, "total": total, "detections": hits}

    def cancel(self) -> None:
        self.cancel_event.set()

def run_video_job_worker(job: VideoJob, service: VisionService) -> None:
    job.status = "processing"
    try:
        # raise_on_cancel=False returns VideoInspectionResult(status='cancelled') and unlinks partial outputs
        res = service.detect_video(
            video_input=job.source_path,
            progress_callback=job.on_progress,
            cancel_event=job.cancel_event,
            raise_on_cancel=False
        )
        job.result = res
        job.status = res.status  # 'completed' or 'cancelled'
    except Exception as e:
        job.status = "failed"
        job.error = str(e)
    finally:
        # Clean up temporary raw upload if needed
        if os.path.exists(job.source_path) and job.status in ("completed", "cancelled"):
            pass # Keep or purge raw upload per Shakib's database retention policy
```

Job Lifecycle & API Routes to Expose

- POST /api/detect/video: Accepts MP4/AVI, assigns UUID4 job_id, starts thread, returns HTTP 200 with status queued.
- GET /api/detect/video/{id}/progress: Returns current percentage, processed frames, and status for UI progress bar.
- POST /api/detect/video/{id}/cancel: Triggers job.cancel(); halts inference loop immediately and unlinks partial video.
- GET /api/detect/video/{id}/result: Returns final damage breakdown, FPS, and streaming URL /outputs/.mp4.

5. Copy-Ready Code Examples for FastAPI & React

FastAPI Global Exception Handlers (Mapping CV Errors to HTTP)

Register these handlers in backend/app/main.py (each handler registers a distinct class or base hierarchy; tuples are not accepted by FastAPI):

```
from fastapi import FastAPI, Request
from fastapi.responses import JSONResponse
from app.services.vision.errors import (
    VisionServiceError, VideoNotFoundError, UnsupportedImageFormatError,
    UnsupportedVideoFormatError, ImageValidationError, VideoValidationError,
    JobCancelledError, JobTimeoutError, ModelLoadError, ModelUnavailableError
)

def register_exception_handlers(app: FastAPI) -> None:
    @app.exception_handler(VideoNotFoundError)
    async def not_found(req: Request, exc: VideoNotFoundError):
        return JSONResponse(status_code=404, content={"error": "Not Found", "detail": str(exc)})

    @app.exception_handler(JobCancelledError)
    async def cancelled(req: Request, exc: JobCancelledError):
        return JSONResponse(status_code=409, content={"error": "Conflict", "detail": "Job was cancelled."})

    @app.exception_handler(JobTimeoutError)
    async def timeout(req: Request, exc: JobTimeoutError):
        return JSONResponse(status_code=504, content={"error": "Gateway Timeout", "detail": str(exc)})

    @app.exception_handler(ImageValidationError)
    async def img_err(req: Request, exc: ImageValidationError):
        status = 400 if isinstance(exc, UnsupportedImageFormatError) else 422
        return JSONResponse(status_code=status, content={"error": "Image Validation Error", "detail": str(exc)})

    @app.exception_handler(VideoValidationError)
    async def vid_err(req: Request, exc: VideoValidationError):
        status = 400 if isinstance(exc, UnsupportedVideoFormatError) else 422
        return JSONResponse(status_code=status, content={"error": "Video Validation Error", "detail": str(exc)})

    @app.exception_handler(VisionServiceError)
    async def cv_err(req: Request, exc: VisionServiceError):
        status = 503 if isinstance(exc, (ModelLoadError, ModelUnavailableError)) else 500
        return JSONResponse(status_code=status, content={"error": "Vision Subsystem Error", "detail": str(exc)})
```

Complete Compilable React HTML5 Video Player Component

Type-safe React component with responsive styling and Faststart MP4 stream support:

```
import React from 'react';

interface RoadVideoPlayerProps {
  annotatedVideoUrl: string; // e.g. "/outputs/Japan_000000_annotated_a1b2c3d4.mp4"
  className?: string;
}

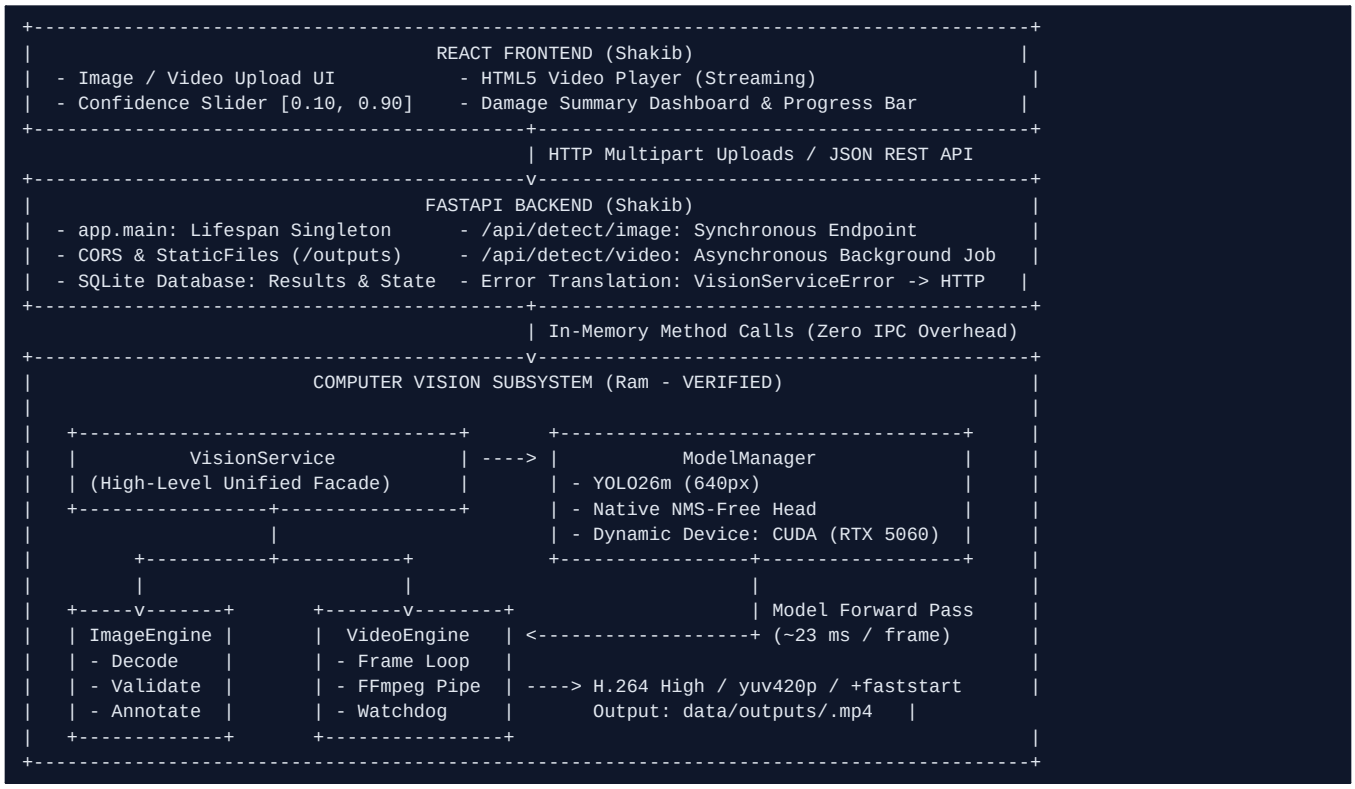
export const RoadVideoPlayer: React.FC<RoadVideoPlayerProps> = ({
  annotatedVideoUrl,
  className = "w-full rounded-lg shadow border border-slate-700",
}) => (
  <div className="relative overflow-hidden bg-slate-950 rounded-lg p-2">
    <video controls playsInline preload="metadata" className={className} src={annotatedVideoUrl}>
      <source src={annotatedVideoUrl} type="video/mp4" />
      <p className="text-slate-400 p-4 text-sm">Your browser does not support HTML5 MP4 video playback.</p>
    </video>
  </div>
);
```

6. Engineering Ownership Demarcation Matrix

To ensure clean collaboration between separate repositories without stepping on code ownership, the boundary between Ram and Shakib is strictly defined:

Subsystem Component	Lead Owner	Current Status	Description & Responsibility
YOLO26m Model & Weights	Ram	**COMPLETE**	models/weights/yolo26m.pt (SHA-256 verified)
Computer Vision Pipeline	Ram	**COMPLETE**	backend/app/services/vision/** (All 14 defects resolved)
FFmpeg Stream Encoding	Ram	**COMPLETE**	H.264 High profile, yuv420p, Faststart moov placement
CV Unit & Regression Tests	Ram	**COMPLETE**	backend/tests/** (94 / 94 tests passing)
FastAPI Application & Lifespan	Shakib	**PENDING**	backend/app/main.py (App startup, CORS, static mounts)
REST API Endpoints	Shakib	**PENDING**	POST /api/detect/image, POST /api/detect/video, progress routes
SQLite Database & Schema	Shakib	**PENDING**	Inspections table, detection records, timestamps, SQLite models
Background Job Orchestration	Shakib	**PENDING**	Thread pool or Celery worker for async video processing
React Dashboard Frontend	Shakib	**PENDING**	Upload forms, bounding box overlays, progress bar, video player
End-to-End System Integration	Joint (Ram + Shakib)	**READY TO START**	Joint acceptance testing and browser-to-GPU verification

7. End-to-End System Architecture Diagram



8. Complete Integration Acceptance Checklist

ID	Integration Verification Task	Owner	Expected Verification Result	Status
CHK-01	Model Checkpoint File & Hash	Ram	SHA-256 matches 1BC41287...D233 exactly	**DONE**
CHK-02	Automated Test Suite Execution	Ram	94 / 94 unit/regression tests pass in < 12s	**DONE**
CHK-03	Four Canonical Damage Classes	Ram	D00, D10, D20, D40 verified across all CV modules	**DONE**
CHK-04	GPU Inference Latency Verification	Ram	Forward pass < 30 ms (RTX 5060: 23.1 ms)	**DONE**
CHK-05	GPU Video Throughput Verification	Ram	Processing speed > 50 FPS (RTX 5060: 60.9 FPS)	**DONE**
CHK-06	Faststart MP4 Header Verification	Ram	moov atom at byte 32 before mdat	**DONE**
CHK-07	FastAPI Lifespan Startup Event	Shakib	VisionService loads on startup, status returns 200	**PENDING**
CHK-08	Synchronous Image API Route	Shakib	POST /api/detect/image returns structured JSON + URL	**PENDING**
CHK-09	Invalid Input Error Rejections	Shakib	0-byte files return 400; corrupt images return 422	**PENDING**
CHK-10	Background Video Detection Job	Shakib	POST /api/detect/video returns job_id and status queued	**PENDING**
CHK-11	Video Progress Polling Endpoint	Shakib	GET /api/detect/video/{id}/progress returns % and FPS	**PENDING**
CHK-12	Video Mid-Stream Cancellation	Shakib	POST /cancel halts job and unlinks partial outputs	**PENDING**
CHK-13	Browser Video Playback in React	Shakib	Annotated MP4 streams instantly in HTML5	**PENDING**
CHK-14	SQLite Metadata Persistence	Shakib	Damage counts, timestamp, media paths saved to DB	**PENDING**
CHK-15	End-to-End Image Demonstration	Joint	User uploads image in UI, sees defect boxes & counts	**JOINT**
CHK-16	End-to-End Video Demonstration	Joint	User uploads video in UI, sees progress & streams output	**JOINT**

9. Fastest Sequential Integration Path

To achieve a working full-stack demonstration in the shortest possible time, follow this strict 8-step critical path:

- **Step 1: Setup & Import Verification:** Ensure virtual environment has requirements.txt installed. Run python -c "from app.services.vision import VisionService; s = VisionService(); s.initialize(); print(s.get_status())" to verify model loads on GPU.
- **Step 2: FastAPI Lifespan Wiring:** Add VisionService singleton initialization to app.main lifespan. Mount /outputs to serve data/outputs via StaticFiles.
- **Step 3: Image Detection API Endpoint:** Implement POST /api/detect/image using the copy-ready snippet in Section 5 with run_in_threadpool. Test with curl using Japan_000000.jpg.
- **Step 4: React Image Upload Component:** Build minimal file picker and image viewer. Display annotated image and damage count cards. (Milestone: First working browser demo!).
- **Step 5: Video Detection Background Job:** Implement POST /api/detect/video and background worker using threading.Event. Wire progress callback.
- **Step 6: Video Progress & Cancellation Routes:** Expose GET /api/detect/video/{id}/progress and POST /cancel. Test with sample video.
- **Step 7: React Video Player Integration:** Embed RoadVideoPlayer component pointing to annotated video URL. Confirm instant streaming without buffering delay.
- **Step 8: SQLite Persistence & History:** Save inspection summaries to SQLite database. Render history table in React dashboard.

10. Architectural Limitations, Policy Constraints & Warnings

- **Full Application Integration is Not Yet Executed:** Ram's repository houses the Computer Vision subsystem. Shakib's application layer (FastAPI routes, SQLite DB, React UI) has not yet been connected or executed end-to-end. Full application readiness remains pending until this integration occurs.
- **Shakib's Independent Repository Uninspected:** Shakib is developing his application layer in a separate repository. Interfaces must adhere strictly to Section 3 of this document to prevent interface divergence.
- **Confidence Policy & Range Clarification:** Both backend/app/config.py and vision config define DEFAULT_CONFIDENCE = 0.25, MIN_CONFIDENCE = 0.10, MAX_CONFIDENCE = 0.90. The CV engine enforces clamp-and-warn on [0.10, 0.90] to prevent runtime crashes. However, Shakib's API layer MUST reject requests outside [0.10, 0.90] with HTTP 422 Unprocessable Entity rather than relying on silent engine clamping. Ram's library code does not implement HTTP 422 boundaries.
- **Operational Frame Cap (MAX_VIDEO_FRAME_CAP = 3600):** Videos exceeding 60 seconds or 3,600 frames are rejected with VideoFrameCountExceededError (HTTP 422) to prevent memory exhaustion on the host.
- **832px Model Resolution is Experimental:** The 832px experiment evaluated in Stage 4 showed a 69% increase in latency without significant mAP gains. The production checkpoint yolo26m.pt is trained and calibrated strictly for 640px.
- **Held-Out Test Split Isolation:** The held-out test split (test/ split) is strictly reserved for independent final evaluation. It must never be used for hyperparameter tuning, confidence threshold selection, or iterative prompt engineering.

11. Direct Collaborative Message to Shakib

Hey Shakib,

I'm excited to hand off the RoadVision Computer Vision Subsystem to you! Everything on the AI/CV side is 100% complete, fully hardened, and verified on real hardware.

Here is what is ready for you right now:
1. The production model (models/weights/yolo26m.pt) is trained, verified, and running on the RTX 5060 GPU with an inference forward pass of ~23 ms per image and over 60 FPS for full video processing.
2. All 14 defects from earlier audits have been resolved. The full test suite of 94 automated tests passes in under 9 seconds.
3. The video engine encodes web-optimized H.264 MP4 with the Faststart moov atom at the start of the file, meaning videos will stream in React immediately without waiting for download.
4. The service facade is clean: call vision_service.detect_image(...) and vision_service.detect_video(...) directly.

What I suggest you implement first:
Start with Step 1 and Step 2 from Section 9: wire VisionService into your FastAPI lifespan and build POST /api/detect/image using run_in_threadpool. Once that's up, wire up the basic React image upload page. Seeing the first road damage bounding boxes render in the browser will give us a huge milestone win!

Whenever you're ready to run joint acceptance tests against the API, let me know and we will knock out CHK-07 through CHK-16 together.

Best regards,
Ram (Senior Computer Vision Engineer)